

Smart Breadboard Diagnostic Trainer

A web replacement for the hardware breadboard trainer. Students build a circuit in a 3D workspace; the system extracts the netlist they actually built, sets it against the instructor's captured reference, and names the difference — which pin, which row, which rail. Two documents follow: one written for a technical reader, one for a general one.

DOCUMENT

Version 1.0

DATE

September 2026

DOMAIN

Electronics laboratory instruction

BUILD STATUS

Phases 0–5 complete

Document 1 · Technical overview

Light technical vocabulary. For the systems chapter and the defence panel.

01

TITLE

Smart Breadboard Diagnostic Trainer: a web-based circuit construction and fault diagnosis system for electronics laboratory instruction

Short title: Breadboard Trainer.

The Smart Breadboard Diagnostic Trainer is a web application that replaces the physical breadboard trainer used in electronics laboratory classes. Students assemble a circuit on a full-size solderless breadboard rendered in 3D inside the browser — placing resistors, LEDs, capacitors, transistors, integrated circuits and jumper wires into tie-points exactly as they would on real hardware.

The difference from a drawing tool is what happens after the circuit is built. The system reads the board **electrically** rather than visually. It walks every occupied tie-point, merges the ones that are physically bonded into a single electrical **node**, and produces a **netlist** — a formal list of which component terminal is connected to which node. That netlist is compared against a **golden netlist**: the reference circuit the instructor built once in Learn Mode and saved with the exercise.

Because the comparison happens on the netlist and not on the picture, a student who builds the correct circuit on row 20 instead of row 12 is marked correct. Only genuine electrical differences register. When a difference is found the system does not return a score — it returns a diagnosis in the vocabulary of the board, together with a guiding question that leaves the isolation work with the student.

⚠ OPEN CIRCUIT · D1

D1 cathode never reaches ground. The jumper sits on the far side of the centre channel, so row 14 and row 15 are two nets rather than one.

Which side of the channel does that jumper need to be on?

A second analysis path runs alongside the comparator. A DC circuit solver based on **modified nodal analysis** evaluates the built circuit to answer behavioural questions — does the LED actually conduct, is the current within range, is a node floating — so faults that are electrically valid but functionally wrong are still caught.

Every student action is logged as a timestamped event. Those events drive per-class analytics for instructors, and they supply the research component of this capstone with its measurable outcomes.

Architecture at a glance

LAYER	TECHNOLOGY	RESPONSIBILITY
Client	React 19 + TypeScript, Vite	All screens, the 3D workspace, and the entire diagnostic engine
3D rendering	Three.js via React Three Fiber	Breadboard, components, fault highlighting
Client state	Zustand	Session, board state, preferences
Styling	Tailwind CSS, locked token palette	Consistent, accessible presentation
API	Hono, serverless functions	Persistence and the entire security boundary
Database	Neon (serverless PostgreSQL)	Users, classes, exercises, attempts, events
Migrations	Drizzle ORM + drizzle-kit	Schema as code, versioned migrations
Authentication	Better Auth, self-hosted	Sessions, verification, password reset
Validation	Zod contracts shared by both tiers	One definition of every request and response
Testing	Vitest, Testing Library, Playwright	Unit, API and real-browser suites
Hosting	Vercel and Neon	Single-origin deployment

Three decisions that define the system

1. The diagnostic engine runs in the browser, not on the server. Feedback must arrive while the student's hand is still on the component. A round trip to a serverless function introduces cold-start latency that would make live diagnosis feel broken. The API stores and retrieves; it never diagnoses.

2. The browser never holds a database credential. Neon is only PostgreSQL — no client SDK, no row-level policy layer that can be safely exposed to a browser. That forces an API tier into the architecture, and that tier becomes the entire security boundary. The consequence is the important part: because no client ever queries the database, it is *structurally impossible* for a student client to receive a golden netlist.

3. The board logic is pure. Hole addressing, node collapsing, netlist extraction and the solver are forbidden from importing React or Three.js. They are plain TypeScript. That is what allows

thousands of headless fault cases to run in seconds, which is what makes the accuracy figures reproducible on demand.

A. Student features

Account and enrolment

- Registration with email verification, sign-in with a remembered session, password reset and change-password. Sessions ride in httpOnly, Secure, SameSite=Lax cookies — no token is placed in browser storage, so a cross-site scripting bug cannot hand over an account.
- A three-step first-run onboarding: confirm name and student number, join a class by six-character code, and a short orientation screen.
- Settings for name, password, language and accessibility. Preferences are stored server-side against the profile, so they follow the student to any lab machine.

Dashboard and exercise library

- Enrolled classes, with per-class completion counted from the student's own exercise list.
- A featured exercise showing the board it builds, reading **Resume** only when an attempt is genuinely still open and **Start** otherwise.
- The full assigned library as rows — unfinished first, finished last — so the screen behaves identically at one exercise and at twenty.
- An exercise brief per exercise: objective, bill of materials, difficulty, schematic, attempt history.
- Designed empty states and skeleton loaders on every list, so nothing is a blank region and no layout shifts when data arrives.

The 3D workspace

- A full-size solderless breadboard with a correct tie-point grid, centre channel, four power rails and printed row labels. Hole geometry is merged into a single instanced mesh so the scene holds 60fps on integrated graphics.
- An orthographic camera with constrained orbit controls — the view cannot reach a below-board or confusing angle.
- Drag from the bill-of-materials tray onto the board with mathematical hole snapping. Two-lead parts are placed hole by hole with a live lead-path preview; DIP integrated circuits straddle the centre channel with orientation and correct pin-to-hole mapping.
- Select, move, rotate and delete, with a full undo/redo stack.
- Placement is refused with a stated reason — occupied hole, span too long, invalid IC position — never silently.
- Board state is serialised on every change, so refreshing mid-build restores the exact board.
- A WebGL-unsupported fallback screen for machines that cannot render the workspace.

Diagnosis and feedback

- **Tier 1 — intrinsic rules.** Faults detectable without the reference circuit, evaluated on every placement: a component straddled across one node, a non-IC bridging the centre channel, an IC not straddling it, a floating lead, a self-jumper, a direct supply-to-ground short, rail continuity wrongly assumed across the mid-board gap, and reversed polarity on a polarised part. Each rule is a pure, individually unit-tested function.
- A **construction-progress heuristic** suppresses open-circuit and floating-lead warnings during normal building — a dangling lead is a fault only once the student marks a subcircuit complete or leaves it untouched for eight seconds. Without this the tool nags, and a nagging tool is abandoned.
- **Tier 2 — the comparator.** Student and golden netlists are matched component-to-component on type and value, then compared by graph edit distance under a depth-limited A* search capped at four edits. Each edit maps to a named fault: an added edge is an open circuit, a deleted edge an extra wire, a moved endpoint a misplaced lead, an adjacent move an off-by-one row, a swapped pair reversed polarity, an attribute change an out-of-tolerance value. Beyond the cap the system reports “substantially different” rather than guessing.
- Only the most upstream fault is reported when several are present, so the student fixes causes rather than symptoms.
- **The DC solver.** Modified nodal analysis with Gaussian elimination; stamps for resistors, DC sources, diodes and LEDs, with capacitors open at DC. A singular matrix is caught and mapped to a floating-node fault rather than crashing.
- **Fault presentation.** The offending strip pulses in 3D while the rest of the board dims. A feedback card carries the fault class, a plain-language cause and a guiding question; a “show me” button focuses the camera on the row.
- A **hint ladder** with three deliberate steps — guiding question, narrowed hint, explicit correction. Each requires a click and each is logged, which is what makes “resolved without help” a measurable quantity.
- Every fault indicator carries an icon and a text label. **No fault is ever communicated by colour alone**, anywhere in the application.

Offline capability

- Installable as a Progressive Web App on Android and desktop.
- Assigned exercises and their metadata are cached, so an exercise can be completed with the network disabled after first load.
- Attempts and events queue in IndexedDB offline and synchronise on reconnect with no loss and no duplication, behind a visible connection indicator.

Accessibility and language

- English and Filipino throughout. The catalogue is typed such that a missing translation key is a **build error**, not a silent fallback to English — the only way the claim “no hardcoded English remains” can honestly be made. Technical vocabulary (netlist, breadboard, resistor, node, rail) stays in English in both catalogues, because Philippine electronics engineering is taught

in English and inventing Tagalog equivalents would produce a vocabulary that matches nothing the student meets again.

- High-contrast, reduced-motion and larger-text toggles, wired to real CSS and to the 3D scene's camera easing.
- Every interactive element has a visible keyboard focus ring; the application is fully operable by keyboard.
- The state palette is verified under simulated deuteranopia using a real colour-matrix filter built into the internal styleguide route, so the claim is checkable rather than asserted.

B. Instructor features

- **Class management.** Create classes with an auto-generated six-character join code; view and manage the roster; remove students; regenerate the code.
- **Exercise authoring.** Title, objective, difficulty, bill of materials, schematic upload and a publish toggle. Uploads use a presigned URL issued by the API, so storage credentials never reach the browser, and content type and size are validated server-side before the URL is issued.
- **Learn Mode** — the feature the diagnostic premise rests on. The instructor builds the reference circuit once in the same 3D workspace students use, previews the extracted netlist in readable form, and saves it as the exercise's golden netlist. Capture is validated: floating leads, shorts or unused bill-of-materials components are refused with a specific reason. Re-capture increments a version and warns about existing attempts.
- **Analytics.** Per class: most common fault classes, average fault isolation time, self-resolution rate, students flagged as struggling. Per student: an attempt drill-down with a fault timeline. Everything exports to CSV.

C. Security and integrity

The golden netlist reaching a student client would invalidate the premise of the system, so it is defended in depth rather than once:

1. **Column projection.** Two select projections exist for exercises — one omitting the golden netlist, one including it. Student-facing handlers may only use the former.
 2. **A serializer whitelist,** applied to every student-bound response object.
 3. **A response tripwire** that scans outbound student JSON for a golden-netlist key at any depth and refuses the response if one is found.
 4. **A build-time bundle check** that greps the compiled JavaScript to prove no golden-netlist-shaped string and no database connection string ships to the browser. It has already caught two real leaks — a barrel import dragging an instructor schema into the student bundle, and a development-only guard surviving minification as a property read.
- Session middleware on every route; role middleware on every instructor route; explicit ownership assertions in every handler taking an ID, so one student cannot read another's

attempt.

- **Self-registration cannot produce an instructor account.** The role requires an invite code and fails closed — with no code configured, no instructor account can be created at all. Without this, any student could read every golden netlist by registering as faculty.
- Rate limiting on sign-in and password reset.
- Single-use verification and reset links, with expired links distinguished from forged ones so a resend button appears only when that is the right answer.
- Sign-in failure messages are deliberately silent about which half was wrong, so the endpoint is not an account-existence oracle.

D. Evaluation harness

Built as a first-class feature, because the results chapter depends on it.

- A seeded corpus of **150 fault fixtures** covering every fault class in proportion to its expected frequency.
- At least **20 negative cases** — correct circuits deliberately built on different rows, which must produce zero faults. This is the headline metric.
- A headless runner that loads a fixture, runs the comparator and compares the result to the expected diagnosis.
- A report generator producing localisation accuracy, classification accuracy, false-positive rate and a per-class confusion matrix, suitable for direct inclusion in the appendix.
- Wired into continuous integration, so accuracy is recomputed on every push and cannot silently regress.

Fault localisation accuracy	≥ 90%
Fault classification accuracy	≥ 85%
False positives on correct builds	≤ 5%
ISO/IEC 25010 weighted mean	≥ 4.20

Primary — the electronics student

A second-year electronics or electrical engineering student in a laboratory course, typically 18 to 21, on a school computer or a mid-range laptop with integrated graphics. Competent with a browser; not a developer.

What they need is specific. On a physical breadboard, a circuit that does not work presents as *nothing happening* — an LED that stays dark says that something is wrong and nothing about what. The productive skill in laboratory work is fault isolation, and it is exactly the skill a silent breadboard cannot teach, because there is no feedback loop. The student asks the instructor, waits, and is told the answer — which teaches them how to get answers rather than how to find faults.

This system closes that loop. The student gets an immediate, specific, non-punitive statement of what is electrically different about their circuit, phrased as a question they can act on. The hint ladder makes the answer available but never free, so isolating the fault remains their work.

Secondary needs the design accounts for: unreliable campus network access (hence offline capability), shared lab machines (hence server-side preferences), a bilingual environment (hence English and Filipino), and colour vision deficiency, which affects roughly one man in twelve — a significant proportion of an electronics cohort, in a domain that traditionally signals everything with red and green.

Primary — the laboratory instructor

A faculty member teaching one or more sections of thirty to forty students. Their constraint is arithmetic: one instructor cannot inspect forty breadboards in a two-hour session, so most students spend most of the session waiting or stuck.

The system gives them three things. Learn Mode lets them define the correct answer once, by building it, rather than by writing a specification. The diagnostic engine absorbs the routine faults — the missing jumper, the reversed LED, the resistor bridging the channel — which frees their attention for students who are genuinely lost. And the analytics turn a class into data: which fault classes recur, which exercises are too hard, which students are struggling before they say so.

Secondary — the department and the researcher

The department gains a laboratory activity that runs without consumable hardware, without damaged components, and outside scheduled hours. The cost of an additional student is effectively zero. For the research component, the telemetry that drives instructor analytics also produces the measurements: fault isolation time per fault, self-resolution rate and hint escalation depth, all computable from raw event timestamps and auditable against the raw log.

Who this system is not for

Stated deliberately, because scope discipline is what protects the delivery date.

- It is **not** a general electronics CAD or PCB design tool. The board is a solderless breadboard and only a solderless breadboard.
- It is **not** an AC or transient simulator. Analysis is DC only, and that is written into the scope.
- It is **not** an autograder. It diagnoses and teaches; the grade remains the instructor's.
- Generative AI appears nowhere in the diagnostic path. Every diagnosis is produced by a pure, unit-tested, deterministic function, which is what makes the accuracy figures meaningful and the results reproducible.

Implementation status

Stated honestly, because a documentation section that overclaims is the first thing a panel will find.

PHASE	SCOPE	STATUS
0	Project setup, tooling, single-origin dev proxy	Complete
1	Design system, tokens, 16 primitives, styleguide route	Complete
2	Database schema, migrations, API foundation, security middleware	Complete
3	Authentication, verification, reset, role guards	Complete
4	Onboarding, class join, profile and accessibility settings	Complete
5	Student dashboard and exercise library	Complete
6	Instructor shell and exercise authoring	Planned
7–8	Workspace shell, polish, usability pre-test	Planned
9–12	Board geometry, placement, netlist extraction, Learn Mode	Planned
13–16	Tier 1 rules, comparator, fault presentation, DC solver	Planned
17–19	Telemetry, instructor analytics, offline and PWA	Planned
20–21	Evaluation harness, pilot and ISO/IEC 25010 evaluation	Planned

Verification as of this document: the production build compiles with zero TypeScript errors across the application, API, tests and configuration; the linter reports warnings only; **128 automated tests pass** across nine files against a real PostgreSQL instance; and **23 end-to-end tests pass in a real browser**, covering registration through sign-out, the reset link, the role guard, the instructor gate, both languages, reduced motion, and a check that nothing credential-shaped reaches browser storage.

Three checklist items remain unticked across the completed phases. **Blocked on accounts** — all three wait on cloud services that have not yet been provisioned (the hosted database, the deployment platform and the code repository), not on unwritten code.

Evaluation methodology

The system is evaluated on two axes.

Technical accuracy is measured by the evaluation harness against the 150-fixture corpus, and reported as localisation accuracy, classification accuracy, false-positive rate and a per-class confusion matrix.

Product quality is measured against **ISO/IEC 25010** across functional suitability, performance efficiency, usability, reliability and portability, with a target weighted mean of ≥ 4.20 .

A pilot with at least 30 student participants runs the same exercises under two conditions: an experimental group with diagnostics enabled, and a control group whose feedback panel reports pass or fail only. The comparison of fault isolation time between the two groups, with a test of statistical significance, supports the project's central efficiency claim.

A formative usability pre-test with five students precedes all of it, run against the student shell before any diagnostic logic exists — which is the reason the application was deliberately built as a complete, usable, empty shell first.

Document 2 · General overview

No technical background assumed. For readers outside electronics and computing.

01

TITLE

Smart Breadboard Diagnostic Trainer: a practice laboratory for electronics students that explains what went wrong

Short title: **Breadboard Trainer.**

The problem, in plain terms

Electronics students learn by building small circuits — a battery, a light, a few components, connected by wires. They build them on a **breadboard**: a small plastic board covered in tiny holes, where pushing a wire into a hole connects it to whatever else is in the same row. It is the electronics equivalent of a practice sketchpad. Nothing is glued or soldered, so a circuit can be built, taken apart and built again.

The problem is what happens when the circuit does not work.

A wrong circuit does not announce itself. The light simply does not turn on. There is no error message, no red underline, no hint. The student is left staring at a board of forty wires with no idea which one is the mistake — and the mistake is usually tiny: a wire one hole too far to the left, a component inserted backwards, a connection that was never made at all.

So the student raises a hand and waits. The instructor, who has thirty to forty other students in the same two-hour session, eventually arrives, glances at the board and points at the wire. The student fixes it and moves on.

Nobody learned anything from that exchange. The student learned that asking works. What they were supposed to learn — how to find the mistake themselves — is the one thing the session never taught, because there was no way to practise it.

What this system does

The Smart Breadboard Diagnostic Trainer moves the practice board into the computer, and gives it the one thing the real board cannot have: **the ability to explain itself**.

A student opens the exercise in a web browser and sees a realistic breadboard in 3D, which can be rotated and zoomed. They drag components onto it and push wires into holes, in the same way and in the same order as they would on the real thing.

Behind the scenes, the system is not watching what the board *looks* like. It is tracking what is electrically connected to what — the same understanding an experienced instructor has when they glance at a board and immediately see the problem.

The instructor has already built the correct version of that circuit once, inside the same program. The system saved it, not as a picture, but as a record of which connections the correct circuit has. When a student builds their version, the system compares the two records and finds the difference.

If there is a difference, the system does not say *wrong* and it does not give a score. It highlights the exact row on the board where the problem is, and it says something like this:

⚠ BROKEN CONNECTION

The LED never reaches ground. The jumper wire is on the far side of the centre channel, so row 14 and row 15 are two separate connections rather than one.

Which side of the channel does that wire need to be on?

That question is the point of the whole system. The student is told exactly where to look, and then asked to work out the fix themselves. If they cannot, they can ask for a stronger hint, and then for the answer outright — but each step is a deliberate choice, and the system records how far they had to go.

Why the comparison is the clever part

There is one problem that makes this harder than it sounds, and solving it is the technical contribution of this project.

There is no single correct-looking circuit. The same circuit can be built on row 5 or row 30, with the components arranged differently, and be completely and equally correct. Two correct boards can look nothing alike.

A system that compared pictures would fail immediately — it would mark a perfectly good circuit as wrong simply because the student built it further down the board. Students would stop trusting it within one session, and a tool nobody trusts is worse than no tool at all.

This system compares **connections**, not appearances. It asks “is this component's leg connected to the same things it should be connected to?” rather than “is it in the same place?” Two circuits built in completely different positions on the board come out identical, because electrically they *are* identical.

That is why the system can be confident enough to point at a specific wire and say *that one* — and it is why the project sets itself a strict measurable target: when a student builds the circuit correctly but differently, the system must report no problems at all, at least 95% of the time.

What the instructor gets

The instructor sets up an exercise once and lets it run. They build the correct circuit themselves — no writing of specifications, no lists of rules, just building it the way they would on their own bench. The system remembers it and uses it to check every student in the class.

While students work, the system handles the ordinary mistakes that account for most raised hands: the missing wire, the backwards component, the connection in the wrong row. That leaves the instructor free to spend their limited time on the students who are genuinely stuck on the concept rather than on the wiring.

Afterwards, the instructor gets a summary of the whole class: which mistakes came up most often, how long students took to find their own errors, how many worked it out without asking for help, and which students are struggling and have not said so. That summary can be exported to a spreadsheet.

None of this replaces the instructor's judgement. The system diagnoses; the grade stays with the teacher.

For the student

FEATURE	WHAT IT MEANS IN PRACTICE
Realistic 3D practice board	A full-size breadboard that can be rotated and zoomed, with components dragged into place. It behaves like the real thing, including refusing impossible placements.
Nothing is graded while you build	The system stays quiet while the student is still working. It speaks up only once a section is finished, or once a wire has been left dangling for several seconds. It is not designed to nag.
Plain-language explanations	Mistakes are described in the language of the board — which wire, which row, which component — never as a percentage or a mark.
Guiding questions before answers	The first response to a mistake is a question, not a correction. A stronger hint and then the explicit answer are available on request, one deliberate step at a time.
Visual pointing	The row containing the mistake glows on the board while the rest dims, and a button swings the camera round to show it.
Free practice	Students can practise outside laboratory hours, as often as they like. Nothing is consumed and nothing can be damaged.
Work is saved automatically	Closing the browser mid-build loses nothing. Reopening the exercise restores the board exactly as it was left.
Works without internet	Once an exercise has been opened once, it can be completed with the connection down. Everything syncs when the network returns.
English and Filipino	The whole interface is available in both. Technical words such as <i>resistor</i> and <i>breadboard</i> stay in English on purpose, because that is how the subject is taught and examined.
Built for different eyes and hands	Larger text, higher contrast, and reduced motion for anyone who finds animation uncomfortable. Everything works by keyboard alone. Crucially, no problem is ever shown by colour alone — every warning carries a symbol and words as well, so colour-blind students, roughly one man in twelve, are never left guessing.
Installable like an app	It can be added to a phone or laptop home screen and opened like any other application.

For the instructor

FEATURE	WHAT IT MEANS IN PRACTICE
Class setup	Create a class, and the system generates a six-character code. Students type that code once to join.
Class list management	See who has joined, remove a student, or issue a new code.
Exercise authoring	Set the title, the objective, the difficulty, the list of parts allowed, and an optional schematic diagram. Exercises stay hidden from students until deliberately published.
Build the answer once	The correct circuit is defined by building it, not by describing it. The system reads it, shows the instructor what it understood, and saves it as the reference for that exercise.
Validation on saving	If the reference circuit itself has a problem — a loose connection, a short, an unused component — the system refuses to save it and says why. It will not let a broken answer key be published.
Class analytics	The most common mistakes, the average time students take to find their own errors, how many resolved a problem without asking for help, and a list of students who appear to be struggling.
Individual review	A timeline of a single student's attempt, showing what they built, what went wrong, and what they did about it.
Spreadsheet export	All of it exports to a standard spreadsheet file.

Protecting the answer key

There is one thing that would break this system completely: if a student could get hold of the instructor's reference circuit, every exercise would become a copying task rather than a learning one.

This was treated as the central security concern of the project rather than an afterthought, and it is guarded in four independent ways. The reference circuit is filtered out of the information sent to students at four separate points, each one an independent check, so that a mistake at any single point is caught by the others. An automated test runs on every change to the software and inspects the actual program delivered to student browsers, confirming that no trace of the answer key is present. **That test has already caught two genuine leaks during development**, before either one could reach a student.

Related protections:

- Nobody can create a teacher account by simply choosing “teacher” when signing up. A teacher account requires a code issued by the department, and if no code has been

configured, no teacher account can be created at all. Without this, any student could see every answer key just by ticking the wrong box at registration.

- One student cannot open another student's work.
- Passwords are never stored in a readable form, and no login information is ever kept anywhere a malicious web page could reach it.
- Repeated failed sign-in attempts are slowed down automatically.
- Password reset links can only be used once, and they expire.

Proving that it works

A capstone project should not merely claim its system is accurate. This one is built to demonstrate it.

A test collection of **150 deliberately broken circuits** has been prepared, each labelled with the mistake it actually contains. The system is run against all 150 automatically, and its diagnosis is compared with the known answer. It reports how often it pointed at the right place, how often it named the right kind of mistake, and how often it complained about a circuit that was in fact correct.

Twenty of those cases are correct circuits deliberately built in unusual positions. The system must find nothing wrong with any of them. These tests run automatically every time the software changes, so accuracy is re-measured continuously and cannot quietly get worse.

Points at the right place	at least 90%
Names the right kind of mistake	at least 85%
Wrongly flags a correct circuit	no more than 5%

The electronics student — the main user

A second- or third-year college student taking an electronics laboratory course, usually 18 to 21 years old, using a school computer or an ordinary laptop. They are comfortable with a web browser and do not need to know anything about programming.

Right now, their laboratory experience has a gap in it. They are graded on whether the circuit works, but they are never actually taught the skill that decides whether it works: methodically finding the one connection that is wrong. That skill is learned by trying, failing, being told precisely what went wrong, and trying again — a cycle the current laboratory setup makes almost impossible, because the board says nothing and the instructor is busy.

This system gives them that cycle. It can be repeated as many times as they want, outside class hours, without waiting for anyone and without embarrassment. And because the system asks a question before it gives an answer, the student is still the one doing the finding.

The design also assumes some realities of their situation: campus internet that drops out, shared laboratory computers that reset overnight, a bilingual classroom, and the ordinary fact that some of them cannot reliably distinguish red from green.

The laboratory instructor — the second main user

A faculty member responsible for one or more laboratory sections of thirty to forty students, in sessions of about two hours.

Their difficulty is arithmetic rather than pedagogical. Forty boards cannot be inspected by one person in two hours. Most of a session is spent queuing, and most of the instructor's attention goes to mistakes that are trivial to spot and tedious to keep spotting.

This system takes the repetitive half of that work. It answers the routine questions immediately and consistently, so the instructor's time goes to the students who are stuck on the idea rather than the wiring. It also tells them things they currently have no way of knowing: which mistakes this particular class keeps making, which exercise is harder than intended, and which quiet student has been stuck for forty minutes without saying so.

The department — the indirect beneficiary

A laboratory section costs money in components, and components get damaged, lost and used up. A practice board that lives in a browser costs nothing per additional student, cannot be broken, and is available at any hour. It does not replace the physical laboratory — students still need to handle real hardware — but it lets them arrive at the real bench already knowing what they are doing.

What this system deliberately does not do

Worth stating clearly, so that expectations are accurate.

- It is **not** a professional electronics design program. It is a teaching board for classroom circuits, and nothing more.
- It handles **steady, direct-current circuits only** — the kind used in introductory laboratory work. Circuits involving alternating current or rapidly changing signals are outside its scope, and that limit is a deliberate decision rather than an omission.
- It does **not** assign grades. It explains mistakes; the marking stays with the instructor.
- It does **not** replace the physical laboratory. Students still need to touch real components. This is preparation and practice, not a substitute.
- It does **not** use artificial intelligence to work out what is wrong. Every diagnosis comes from fixed, tested electrical rules. This matters for the research: the system gives the same answer for the same circuit every single time, which is what makes its accuracy measurable and the results of this study repeatable by anyone who checks them.

Current status of the project

Stated plainly, so that no part of this document is read as a claim about something that is not yet built.

Built and working today

The complete student-facing application exists as a working program: account creation with email confirmation, secure sign-in, password reset, first-time setup, joining a class by code, the student dashboard with the full exercise library, the exercise briefing screen, the settings screen with its language and accessibility options, and the full visual design system underneath all of it. Both languages are complete. The protections around the answer key and around teacher accounts are in place and independently tested.

Verified by automated testing

The program is checked by **128 automated tests** running against a real database, plus **23 tests driving a real web browser** from account creation through to signing out. All of them pass. These are not manual checks — they run automatically and would fail loudly if anything broke.

Still to be built

The 3D building workspace itself, the circuit reading and comparison engine, Learn Mode for instructors, the class analytics, and the offline capability. These are the remaining planned phases of the project, and each has a written specification and a defined completion checklist.

Waiting on accounts rather than on work

Three items are finished as code but cannot be ticked off yet because they need online accounts that have not been set up — the hosted database, the web hosting service and the code repository. None of them requires further development.

How the project will be judged

Two separate measurements, taken independently, and a third that asks whether it helps.

Does it diagnose correctly? Measured automatically against the 150 prepared test circuits, and reported as three percentages: how often it pointed at the right place, how often it named the right kind of mistake, and how often it wrongly flagged a correct circuit.

Is it a good product? Measured using **ISO/IEC 25010**, an international standard for judging software quality across five areas — whether it does what it should, whether it performs well, whether people can actually use it, whether it is dependable, and whether it runs on the machines it needs to. The target is an average rating of **4.20 out of 5**.

Does it actually help? A trial with at least 30 students runs the same exercises two ways: one group with the explanations turned on, and one group told only whether their circuit passed or failed. Comparing how long each group takes to find their own mistakes is what shows whether the system teaches, rather than merely functions.

Smart Breadboard Diagnostic Trainer · capstone system documentation, version 1.0 · September 2026.
Source documents: docs/system-documentation-technical.md and docs/system-documentation-general.md.